

Dead Harvest

Cronograma de sprints, enlace del prototipo y componentes

Equipo Iron Pixels
Ingeniería de Software II TS359
Universidad del Valle de Guatemala Campus Altiplano
Docente Omar Arreaga
Entrega del 29 de septiembre de 2026

Propósito de la entrega

Presentamos la planificación de Dead Harvest, el acceso al prototipo y el diseño de componentes desarrollado a partir de la clase 10. Reutilizamos el Sprint Planning, el Roadmap y los diagramas anteriores y los complementamos con interfaces, integración incremental y criterios de calidad verificables.

El diseño de componentes define cómo construiremos el juego. Los contratos y pruebas de este documento son especificaciones propuestas; su cumplimiento deberá demostrarse con código y resultados de ejecución. La evidencia de planificación y diseño no acredita por sí sola una implementación terminada.

Lo solicitado	Contenido de esta entrega
Cronograma de sprints	Secciones 1 y 2. Calendario consolidado propuesto y capturas del tablero y Roadmap existentes.
Enlace del prototipo	Sección 3. URL pública, archivo de Figma y recorrido de revisión.
Componentes de la clase 10	Secciones 4 a 9. Catálogo, diagramas, contratos, integración, pruebas y checklist.
Reto de integración	Sección 10. Análisis y texto para la participación en el foro.

Integrantes

Cristian Sahón.

Marcos Sulugüí.

Nahum Albeño.

Gilmar Maldonado.

Wendy Chumil.

Hefer Cotúc.

La coordinación corresponde a Cristian; la consolidación documental, a Marcos; y la validación de calidad, a Hefer según los roles registrados. Las asignaciones técnicas específicas se conservan en el plan de trabajo.

1 Planificación existente y consistencia

El tablero de GitHub Projects y el Roadmap son evidencias de una misma planificación: el primero organiza el trabajo por estado y el segundo muestra su distribución temporal. Se incorporan nuevamente porque la actividad del 29 de septiembre los solicita junto con el prototipo y los componentes.

Fuente	Qué representa	Lectura correcta
Excel de Hefer	65 actividades propuestas con responsables y criterios de terminado.	11 en S1, 14 en S2, 13 en S3, 16 en S4, 7 en S5 y 4 de evaluación.
Tablero capturado	25 elementos: 8 RF, 3 RNF y 14 historias o tareas.	5 están en la columna Sprint 1 y 0 en Done; no representa el desglose de 65 actividades.
Roadmap capturado	Fechas de los elementos del tablero, principalmente septiembre y octubre.	La duración de una barra es planificación; no demuestra que la actividad se haya completado.
PDF del 10 de septiembre	Resumen de avances, prototipo, evaluación UX y distribución del trabajo.	Se conserva como antecedente; esta entrega añade el detalle técnico de la clase 10.

Ajustes de consistencia propuestos

HU-01 Movimiento y HU-05 HUD aparecen en la columna Sprint 1 pero con etiqueta Sprint 0. Deben distinguirse sus tareas de diseño, pertenecientes al Sprint 0, de su implementación. La estamina aparece en Sprint 5 en el tablero, mientras el Excel la ubica en el núcleo; su implementación básica debe anteceder a ataques y esquiva, dejando para el cierre únicamente el ajuste o balance.

El Excel y el tablero también distribuyen de forma diferente armas, IA, oleadas y jefe final. Se propone mantener los códigos HU, RF y RNF originales y asociar las tareas técnicas S1-01 a S5-07 a esas historias, sin sumar historias y subtareas como si fueran el mismo indicador de avance.

El calendario de la sección siguiente es una propuesta consolidada para cubrir el ciclo completo y respetar las dependencias. No acredita cambios ya aplicados en GitHub. Las capturas originales se conservan en el anexo para mantener visible la planificación anterior.

Regla para marcar trabajo terminado

Una tarea de implementación se cierra cuando existe un cambio identificable en el repositorio, revisión de otro integrante, prueba registrada y evidencia de que el flujo anterior sigue funcionando. Un diseño o una fecha vencida no sustituyen esa evidencia.

2 Cronograma consolidado del proyecto

Propuesta de organización para esta entrega. Incluye el inicio del análisis el 2 de agosto, conserva las fases del plan anterior y reserva del 5 al 18 de noviembre para evaluación. Los períodos pasados se muestran como ventanas planificadas, sin declarar ejecución retroactiva.

Etapa y fechas de 2026	Trabajo y salida esperada	Coordinación propuesta
Sprint 0 02 ago al 08 sep	Análisis, requisitos, UML, wireframes, mockups y prototipo. Salida: documentación base y flujo diseñado.	Equipo; Marcos consolida; Wendy y Nahum apoyan el prototipo.
Sprint 1 09 al 22 sep 11 actividades	Escena base, movimiento, vida, estamina, combate, armas y HUD mínimo. Salida: núcleo que pueda probarse.	Cristian y Nahum; Marcos y Hefer en vida y estamina; Wendy en HUD.
Sprint 2 23 sep al 06 oct 14 actividades	IA, oleadas, mejoras y dificultad; pruebas unitarias. Integrar por incrementos desde esta etapa.	Equipo técnico; Hefer coordina pruebas y Nahum apoya IA.
Sprint 3 07 al 20 oct 13 actividades	Integración del flujo, menús, pausa, HUD y audio. Salida: recorrido de partida conectado.	Cristian coordina; Wendy y Nahum integran UI; Hefer verifica.
Sprint 4 21 al 28 oct 16 actividades	Integrar y validar jefe final, balance, RNF y manuales técnico y de sistema.	Hefer coordina QA; Gilmar apoya validación; Marcos documenta.
Sprint 5 29 oct al 04 nov 7 actividades	Build, regresión final, evidencias y entrega del producto. Salida: versión identificada y manuales.	Cristian y Nahum preparan build; equipo realiza revisión final.
Evaluación 05 al 18 nov 4 actividades	Medir y comparar resultados con requisitos; retrospectiva y cierre. No planificar funciones nuevas.	Hefer, Gilmar y equipo; Marcos integra resultados.

Hitos y seguimiento

29 de septiembre: entrega documental. 3 de octubre: demostración de InnovaCon, con el avance real disponible. 4 de noviembre: cierre propuesto de desarrollo y entrega. 5 al 18 de noviembre: evaluación de resultados. El equipo revisará semanalmente tareas terminadas con evidencia, bloqueos y capacidad restante.

3 Prototipo y recorrido de revisión

El prototipo permite revisar las pantallas y la navegación previstas. Es la referencia de interfaz para desarrollar Dead Harvest en Unity. Al momento de esta entrega continúa en refinamiento: el equipo está incorporando mejoras visuales y de audio. Estas mejoras se documentan como trabajo en curso y no se consideran verificadas hasta revisar la versión publicada.

Enlaces del proyecto

Prototipo público de Dead Harvest

https://middle-violin-46821554.figma.site

Archivo editable de Figma del prototipo completo

Paso	Recorrido esperado	Qué observar
1	Menú principal y Cómo jugar	Identificar Nueva Partida, Opciones, controles y regreso.
2	Inicio y Gameplay	Reconocer vida, estamina, arma y número de oleada.
3	Pausa y cambio de arma	Comprobar reanudar, regresar y seleccionar/cancelar sin perder control.
4	Oleada completada y mejoras	Comprender descanso, selección y regreso a la partida.
5	Jefe final y resultados	Distinguir victoria y derrota; revisar estadísticas y reintento.

Mejoras UX que deben comprobarse

Las evaluaciones de Gilmar y Nahum documentan ocho hallazgos: tres altos, tres medios y dos bajos. Priorizar visibilidad de estamina, contexto de pausa y progreso de oleada. Después revisar confirmación de mejoras, cancelación del cambio de arma, cuenta regresiva inicial, ayudas de controles y explicación de la derrota.

El informe de hallazgos existe. La aplicación de todas las correcciones y el uso de componentes o auto-layout en Figma requieren revisión del archivo editable; no se declaran verificados en este documento.

4 Catálogo de componentes

Los componentes siguientes refinan los subsistemas y requisitos vigentes de Dead Harvest. Cada componente agrupa responsabilidades relacionadas y se comunica mediante contratos explícitos. C10 Sistema de Audio concentra música, SFX y ambiente; no se crea un componente separado para cada tipo de sonido. La ERS v0200 se usa como referencia de alcance y trazabilidad (RF-01 a RF-08, RNF-01 a RNF-03).

ID y componente	Responsabilidad y límite	Trazabilidad
C01 Núcleo de Partida	Estados de inicio, ejecución, pausa y resultado. Coordina transiciones; no calcula daño ni dibuja el HUD.	RF-06, RF-07
C02 Interfaz de Usuario	Menús, HUD, resultados y selección. Emite solicitudes y presenta datos; no modifica vida directamente.	RF-05, RF-06, RF-07
C03 Sistema del Jugador	Movimiento, vida, estamina y esquiva. Provee posición y recepción de daño.	HU-01, RF-08
C04 Sistema de Combate	Armas, alcance, ataque y resolución del daño. Solicita el consumo de estamina.	RF-01, RF-02
C05 Enemigos e IA	Creación y estados Patrullar, Perseguir y Atacar. Mantiene vida de enemigos y notifica muertes.	RF-03, HU-08
C06 Gestión de Oleadas	Generación, conteo, descanso y dificultad. Determina cuándo una oleada concluye.	RF-04, HU-10
C07 Sistema de Mejoras	Selección, aplicación y caducidad de modificadores. No sobrescribe los valores base.	HU-06
C08 Entorno y Nivel	Escena, límites, colisiones, puntos válidos de aparición y objetos recolectables del nivel (botiquines). No controla vida ni combate.	HU-01, HU-05, RF-03, RF-04
C09 Sistema de Jefe Final	Fases y patrones especiales. Utiliza combate y comunica su derrota al núcleo.	HU-09
C10 Sistema de Audio	Música/BGM, SFX, ambiente y volúmenes. Reacciona a eventos de partida y gameplay; no decide daño, oleadas ni resultados. Incluye la propuesta sonora en refinamiento del prototipo.	RF-01 (feedback sonoro), Opciones del prototipo

Unity 2D es la plataforma de soporte, no otro componente de reglas del juego. La ERS establece Unity 2022.3 LTS, C#, Input System y UGUI. El guardado persistente permanece fuera del alcance del MVP. Los botiquines, incluidos en la ERS, se modelan como objetos del entorno (C08) cuya recolección solicita recuperación de vida al jugador (C03), sin crear un componente adicional para este alcance.

5 Diagrama de coordinación y oleadas

Vista de dependencias del diseño propuesto. La flecha discontinua apunta del componente consumidor al proveedor de una interfaz. I01 a I06 se especifican en la sección 6; los eventos se detallan por separado para distinguir solicitudes de notificaciones.

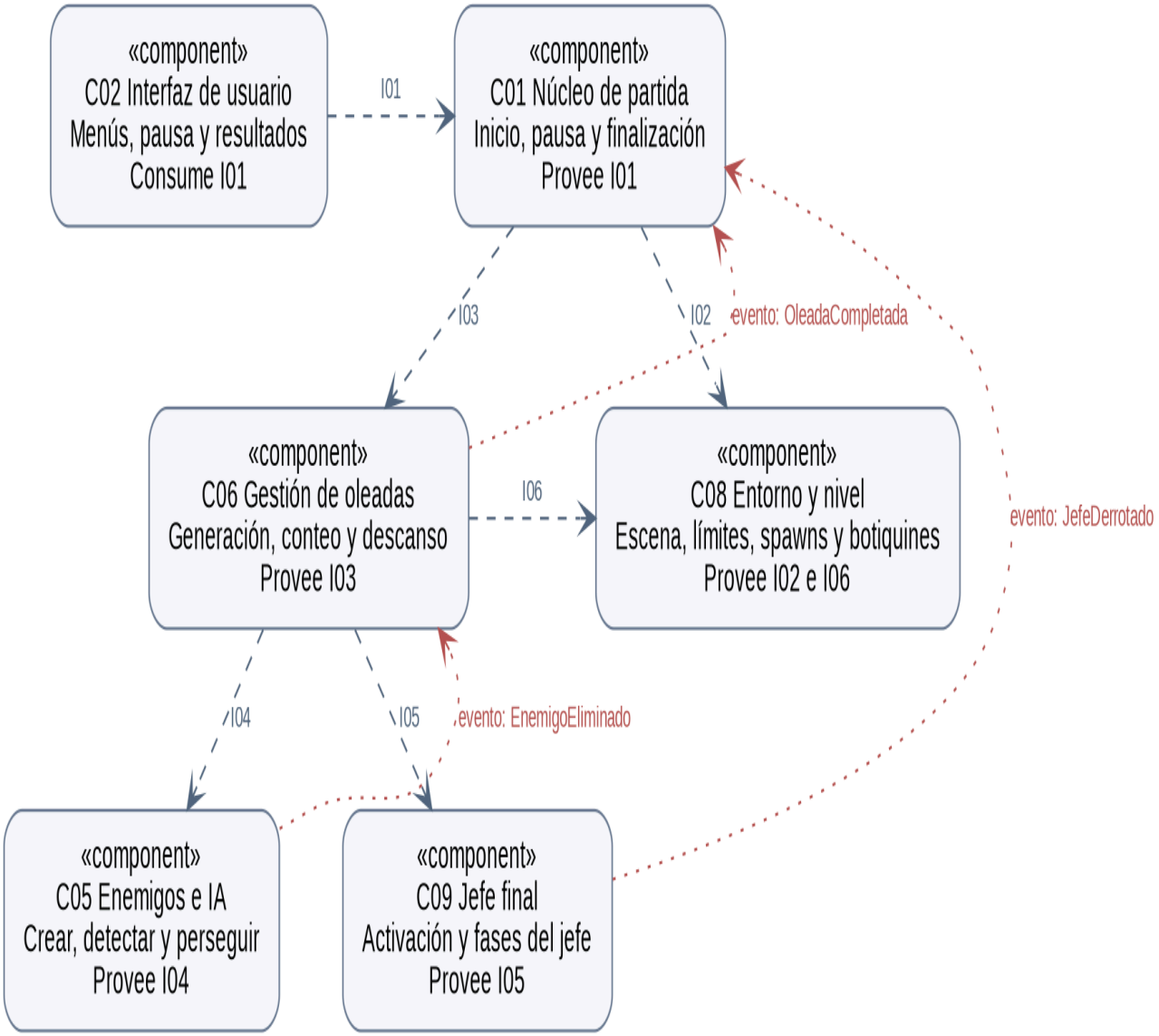


Figura 1. Coordinación y oleadas: Las líneas I01-I06 representan contratos de solicitud. Las líneas rotuladas como eventos representan notificaciones ya ocurridas: EnemigoEliminado, OleadaCompletada y JefeDerrotado. C06 contabiliza enemigos activos y C01 resuelve las transiciones de partida.

Comunicación sin dependencias internas

La UI solicita pausar mediante I01; el núcleo publica el cambio de estado. Oleadas obtiene puntos de aparición por I06 y solicita enemigos a través de I04. Cada enemigo publica su eliminación una sola vez; Oleadas descuenta ese identificador y emite el fin de ronda cuando el conteo llega a cero y ya terminó la generación.

5 Continuación del diagrama de combate

Esta vista amplía las dependencias de jugador, combate, mejoras, UI y audio. Los componentes repetidos conservan el mismo ID y representan el mismo módulo. C10 Sistema de Audio consume eventos de partida y gameplay para reaccionar con BGM, SFX y ambiente, mientras I14 queda reservado al control de volumen desde la UI.

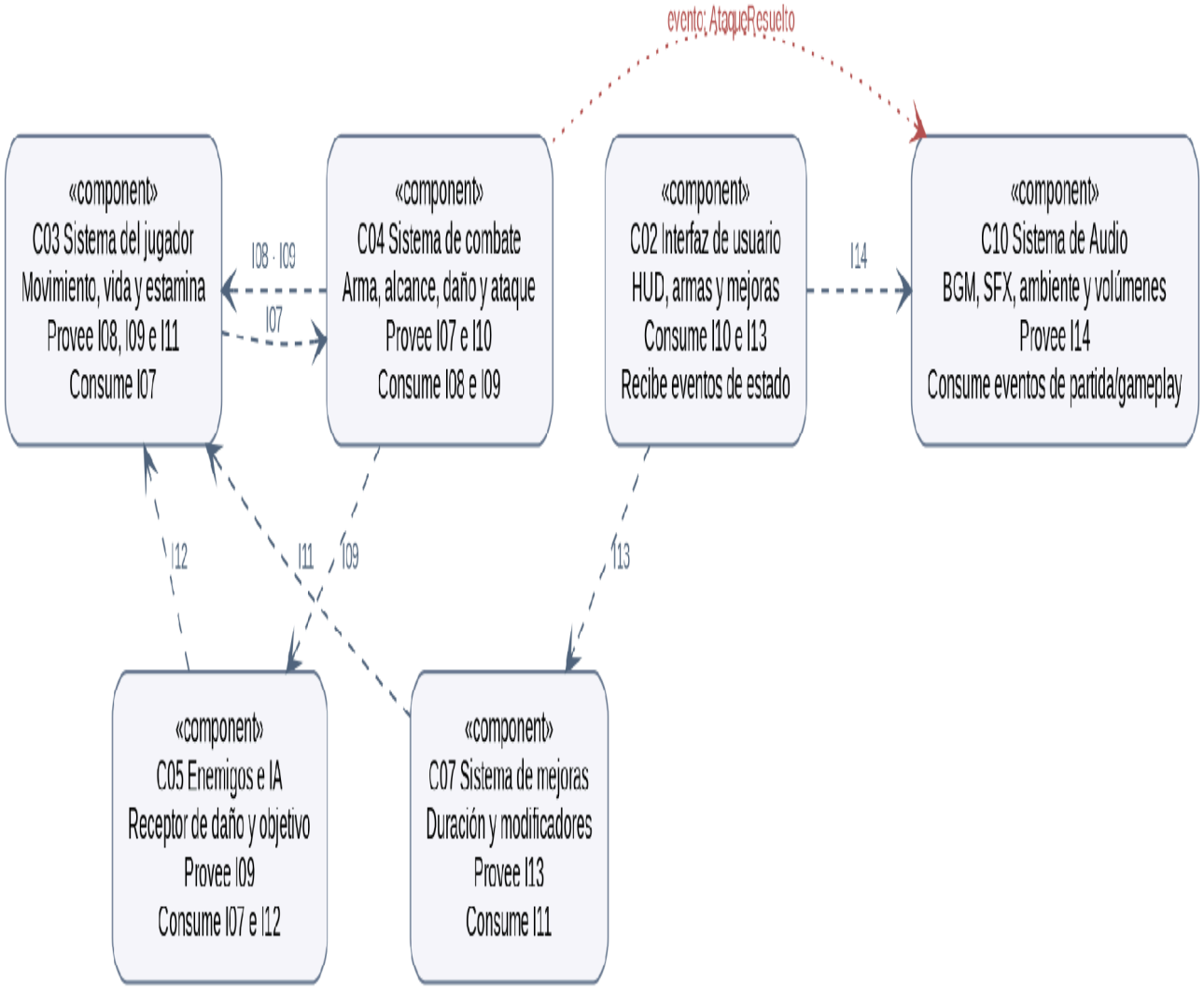


Figura 2. Combate y presentación: I07-I14 representan contratos. Los eventos AtaqueResuelto y EstadoPartidaCambiado permiten activar SFX/BGM sin introducir dependencias de cálculo de daño. C10 no decide reglas de combate ni resultados.

Las flechas expresan contratos del diseño, no interfaces ya programadas. Ambas vistas se incluyen como páginas editables en el archivo Dead_Harvest_Componentes_Interfaces.drawio.

6 Contratos de comunicación

Cada interfaz identifica un proveedor y uno o más consumidores. Los nombres de operaciones son una propuesta de contrato para C#; los tipos concretos se definirán durante la implementación manteniendo estas responsabilidades.

ID y proveedor	Consumidor y operación	Condición y resultado
I01 IPartida C01	C02: Iniciar(), Pausar(), Reanudar(), Reiniciar().	Transición válida según estado. Reiniciar limpia conteos, mejoras y suscripciones de la partida anterior.
I02 INivel C08	C01: PrepararNivel(config).	Informa éxito o error; la partida no comienza hasta disponer de jugador y zona válidos.
I03 IOleadas C06	C01: IniciarOleada(numero), Detener().	Solo en estado Jugando y sin otra generación activa. Detener cancela futuras apariciones.
I04 IEnemigos C05	C06: Crear(tipo, posicion, oleadaId).	Devuelve enemigoId único o fallo. Solo una creación exitosa se registra como enemigo activo.
I05 IJefe C09	C06: Activar(config, posicion).	Una activación por encuentro; informa el identificador del jefe o fallo.
I06 IPuntosAparicion C08	C06: ObtenerPunto(tipo).	Devuelve posición válida dentro del mapa y libre de obstáculo; si no existe, retorna fallo recuperable.
I07 ICombate C04	C03, C05, C09: SolicitarAtaque(atacanteId, direccion).	Valida estado, intervalo y recursos; devuelve aceptado/rechazado. El cliente no aplica daño por su cuenta.

Respuesta ante fallos

Una solicitud rechazada no consume estamina ni causa daño. Si falla una aparición, no se incrementa el contador de enemigos vivos; se registra el fallo y se aplica un reintento limitado. Un fallo persistente detiene la generación y permite reiniciar la partida, evitando una oleada imposible de terminar.

6 Continuación de los contratos

ID y proveedor	Consumidor y operación	Condición y resultado
I08 IEstamina C03	C04: IntentarConsumir(costo); C03 la usa para esquiva.	Consumo atómico si el recurso alcanza. Siempre entre cero y máximo. Regeneración y penalización según RF-08.
I09 IReceptorDanio C03, C05, C09	C04: RecibirDanio(cantidad, origenId, ataqueId).	Daño no negativo. Un objetivo registra como máximo un impacto por ataque; muerte se emite una sola vez.
I10 IArmas C04	C02 o entrada de C03: Equipar(armaId).	Espada, Hacha, Mazo o Lanza. Actualiza daño, alcance y velocidad; publica ArmaCambiada.
I11 IModificadores C03 y C04	C07: Agregar(modificadorId, efecto), Retirar(id).	Cada modificador tiene ID y duración; retirar no altera la base ni elimina otros modificadores.
I12 IObjetivo C03	C05 y C09: ConsultarPosicionYEstado().	Devuelve posición y si sigue vivo; evita acceso a detalles internos del jugador.
I13 IMejoras C07	C02: ConsultarOpciones(), Seleccionar(id).	Solo entre oleadas y una elección válida; publica MejoraAplicada para continuar.
I14 IAudio C10	C02: AjustarVolumen(canal, valor). C10 consume eventos de C01/C04/C06/C09 para seleccionar o transicionar BGM/SFX.	Canales general, música y efectos; valor 0 a 1. El audio reacciona al contexto (menú/cinemática, gameplay, jefe y resultado). Un fallo de audio no bloquea el gameplay.

Reglas de diseño compartidas

El núcleo es dueño del estado de partida; Jugador, de vida y estamina; Combate, de armas y ataques; Oleadas, de sus contadores. La UI lee datos y envía solicitudes. Las dependencias se configuran al crear la escena, sin búsquedas continuas entre objetos durante cada actualización.

Las duraciones de ataques, descanso y mejoras utilizan tiempo de juego: se congelan en pausa. Las suscripciones a eventos se registran y retiran con el ciclo de vida de la escena para impedir efectos duplicados después de un reinicio.

7 Eventos y secuencia de una partida

Los eventos notifican un cambio ya ocurrido. Se publican después de actualizar el estado y transportan datos mínimos; los receptores no vuelven a ejecutar la acción que originó el evento.

Evento	Emisor y receptores	Datos y efecto esperado
EstadoPartidaCambiado	C01 → C02, C03, C04, C05, C06, C07, C09, C10	Estado anterior y nuevo. Pausa congela acciones y temporizadores; C10 selecciona/transiciona el contexto musical sin modificar reglas.
VidaCambiada y EstaminaCambiada	C03 → C02	Actual y máximo; el HUD actualiza barras y avisos.
ArmaCambiada	C04 → C02	armaId y atributos; actualiza selección e indicador.
AtaqueResuelto	C04 → C10 y presentación	ataqueId, origen y resultado. C10/presentación reproducen feedback una sola vez.
EnemigoEliminado	C05 → C06	enemigoId y oleadaId. Se descuenta solo si pertenece al conjunto activo.
OleadaCambiada y OleadaCompletada	C06 → C02, C01	Número, restantes y estado. Al terminar generación y quedar cero vivos, abre descanso/mejoras.
MejoraAplicada	C07 → C01, C02	mejoraId. Confirma selección; el núcleo autoriza continuar cuando corresponde.
JugadorMuerto y JefeDerrotado	C03 o C09 → C01	Causa, identificador y estadísticas. El núcleo resuelve un único resultado final.

Ejemplo de extremo a extremo

El jugador solicita atacar. Combate verifica intervalo y estamina; si acepta, resuelve los impactos a través de I09. El enemigo que llega a cero vida emite EnemigoEliminado. Oleadas descuenta su ID y, si terminó la generación y no quedan enemigos, notifica OleadaCompletada. El núcleo abre el descanso y la selección de mejora; la UI presenta cada transición.

Propuesta para casos simultáneos: si la vida del jugador llega a cero en el mismo paso de simulación que cae el jefe, el núcleo resuelve derrota. Esta regla debe mantenerse en código y en pruebas para evitar dos pantallas finales o estadísticas contradictorias.

7.1 Actualización del componente de audio

Estado de la propuesta. El Scrum Master definió una ampliación de C10 Sistema de Audio basada en una adaptación horror procedural de “Luna de Xelajú”. La propuesta busca conservar una referencia musical reconocible y reinterpretarla como terror psicológico mediante tempo muy lento, desafinación sutil por nota, silencios entre frases, reverberación simulada y refuerzo de octavas graves. Se trata de una especificación del prototipo en refinamiento: debe verificarse en la versión publicada y posteriormente adaptarse a Unity.

Separación interna propuesta dentro de C10: BGM/Música, SFX/Efectos y Ambiente. Esta organización mantiene alta cohesión y no crea componentes nuevos: todos pertenecen a la presentación sonora y reaccionan a eventos sin asumir reglas de gameplay.

Aspecto	Especificación propuesta	Uso previsto
Adaptación musical	“Luna de Xelajú” reconstruida nota a nota y reinterpretada como horror procedural.	Identidad sonora propuesta de C10; pendiente de verificación en la versión final del prototipo y posterior adaptación a Unity.
Cinemática + Menú	Versión completa, aproximadamente 62 s por ciclo, tempo cercano a 28 BPM, drones de pueblo nocturno, drift individual de -0.04 a +0.02 semitonos, silencios de 2–4 s y reverb simulada con delays de 70, 140, 270 y 480 ms.	Crear una atmósfera lenta, incómoda y reconocible.
Gameplay	Misma base musical con capa adicional de tensión; inicio aproximado a los 3 s del spawn.	Acompañar oleadas sin modificar reglas del combate.
Jefe final	Versión acelerada, aproximadamente 35 s por ciclo, con clusters y Eb añadida como nota disonante.	Elevar la tensión del encuentro final.
Título DEAD HARVEST	A los 0.9 s: rumble 28→18 Hz, estática filtrada, dos sawtooth 1047/1108 Hz descendentes, dos pulsos 60→25 Hz y un tono final cercano a 3200 Hz.	Efecto perturbador sincronizado con el frame del título.
Ambientación narrativa	Textos cinemáticos actualizados a El Tablón, Sololá, Guatemala, con referencias al Lago Atitlán y la carretera a Sololá.	Contextualizar la presentación y la ambientación del prototipo; no constituye una regla de gameplay.

Nota: estos valores y comportamientos provienen de la especificación enviada por el Scrum Master. Mientras el prototipo siga recibiendo mejoras de sonido e imágenes, deben presentarse como propuesta/en refinamiento y no como implementación final verificada en Unity.

8 Plan de integración incremental

Se integra un incremento por vez. Cada fila depende de las anteriores; antes de avanzar se repiten sus pruebas. Los responsables se proponen a partir del reparto previo. Todas las pruebas siguientes están pendientes de ejecución y deben registrarse con build o commit, fecha y resultado.

Paso y vínculo	Componentes y prueba de aceptación	Responsables propuestos
01 · S1 HU-01	C08 + C01 + C03. Cargar escena, mover en ocho direcciones y chocar contra límites. Aceptar si no atraviesa obstáculos y la escena inicia sin errores.	Cristian / Nahum; verifica Hefer.
02 · S1 RF-08	C03 vida y estamina. Atacar/esquivar repetidamente y regenerar. Aceptar límites correctos, penalización al agotarse y ausencia de gasto en acciones rechazadas.	Marcos / Hefer; revisa Cristian.
03 · S1 RF-01, RF-02	C04 + C03 contra objetivo de prueba. Verificar daño, alcance, intervalo y las cuatro armas. Un ataque no daña dos veces al mismo objetivo.	Cristian / Nahum; verifica Hefer.
04 · S2 RF-03	C05 + C04 + C03. Probar Patrullar, Perseguir y Atacar con posiciones dentro y fuera de rango. La muerte se notifica una vez.	Nahum / Cristian; verifica Hefer.
05 · S2 RF-04	C06 + C08 + C05. Crear oleada pequeña, eliminarla y repetir. Conteo igual a enemigos activos; descanso y siguiente oleada sin duplicados.	Equipo técnico; verifican Hefer / Gilmar.
06 · S1 a S3 RF-05, RF-06	C02 + C01 y eventos. HUD mínimo desde S1, integración completa en S3. Pausar detiene daño y tiempos; reanudar y reiniciar no duplican eventos.	Wendy / Nahum; verifica Hefer.
07 · S2 a S3 HU-06	C07 + C03 + C04 + C02. Elegir una mejora, comprobar efecto, pausa y caducidad. Reiniciar restaura valores base.	Equipo técnico / Wendy; verifica Hefer.
08 · S3 RF-01 y opciones	C10 + eventos + C02. Verificar SFX de interacción/combate, transición de BGM entre menú/cinemática, gameplay, jefe y resultado, y separación de controles MÚSICA/EFECTOS. Aceptar si volumen cero silencia el canal correspondiente y el gameplay continúa aunque el audio falle.	Nahum / Wendy; verifica Gilmar.
09 · S4 HU-09, RF-07	C09 + C06 + C04 + C01. Probar fases, derrota del jefe y muerte simultánea del jugador. Solo un resultado y estadísticas coherentes.	Cristian; verifican Hefer / Gilmar.
10 · S4 a S5 RNF-01 a 03	Flujo y build completo. Repetir menú, partida, pausa, oleadas y resultado; medir rendimiento, latencia y reconocimiento del HUD.	Hefer / Gilmar; equipo corrige.

8 Evidencias y control de integración

Procedimiento por incremento

El responsable trabaja en una rama vinculada a la tarea. Registra qué contrato cambia y qué requisito atiende. Otro integrante revisa la modificación antes de integrarla. Se ejecuta la prueba específica y la regresión de los pasos anteriores. Si falla, se conserva la última versión funcional y se corrige antes de integrar el siguiente componente.

Se propone Unity Test Framework para lógica determinista y pruebas de escena cuando corresponda. La integración continua automatizada es una mejora futura: no se declara configurada ni ejecutada. La primera meta es disponer de comprobaciones repetibles para cada cambio.

Prueba prioritaria	Caso normal y caso límite	Evidencia requerida
P01 Estamina y daño	Consumo exacto, costo mayor al saldo, daño duplicado, vida igual a cero.	Valores antes/después; resultado de prueba de lógica.
P02 Oleadas	Último enemigo, generación fallida y notificación repetida del mismo ID.	Registro de IDs activos, contador y transición única.
P03 Pausa y reinicio	Pausar durante ataque o mejora; reiniciar varias veces.	Captura o video con temporizadores congelados y valores reiniciados.
P04 Flujo final	Victoria, derrota, resultado simultáneo y volver al menú.	Un resultado final, estadísticas correctas y nueva partida limpia.
P05 Regresión	Repetir P01 a P04 tras cada componente nuevo.	Build o commit probado y lista de comprobaciones aprobadas/fallidas.

Formato de registro de pruebas

Cada ejecución registrará: ID de prueba, requisito, versión o commit, equipo utilizado, precondiciones, pasos, resultado esperado, resultado observado, aprobado/fallido, evidencia, defecto asociado y responsable. Los valores observados se completan al ejecutar; no se sustituyen por resultados esperados.

Criterio de aceptación de un incremento

Contratos respetados, prueba del incremento aprobada, regresión anterior aprobada, errores bloqueantes resueltos y evidencia identificable. El registro en GitHub debe enlazar esa evidencia. Los defectos no bloqueantes se mantienen visibles con responsable y fecha prevista.

9 Checklist de calidad del diseño

Estado al preparar esta entrega. Cumplido documental significa que existe el artefacto de diseño; parcial indica una comprobación o ajuste pendiente; no verificado significa que todavía no hay evidencia de ejecución suficiente para evaluar el código o el juego.

Criterio	Estado	Evidencia y acción
Requisitos y trazabilidad	Cumplido documental	ERS y relación RF/HU. Esta entrega conserva RF-01 a RF-08 y RNF-01 a RNF-03.
Responsabilidad única	Cumplido en diseño	Catálogo con límites por componente. Revisar que el código mantenga esta separación.
Interfaces explícitas	Cumplido en diseño	I01 a I14 y eventos definidos en este documento. Implementación pendiente de verificar.
Alta cohesión y bajo acoplamiento	Parcial	Responsabilidades y contratos definidos; inspeccionar dependencias reales en C#.
Git e historial	Cumplido documental	Repositorio con entregas y organización por semanas. Verificar aparte integración del código Unity.
Commits pequeños y revisión	No verificado	Registrar cambios de código y revisión de otro integrante; no se acredita por disponer de documentos.
Nombres claros y sin duplicación	No verificado	Revisar nombres, clases y reutilización en código. Evitar repetir cálculos de daño y estamina.
Pruebas por incremento	Planificado	P01 a P05 y tabla de integración listos. Ejecutar y conservar resultados.
Evaluación de usabilidad	Parcial	Ocho hallazgos documentados. Verificar correcciones y aplicar prueba RNF-02.
Prototipo accesible	Por comprobar	URL completa incluida. Abrir en sesión sin permisos de editor antes de la entrega.
Consistencia del tablero	Parcial	Cronograma consolidado propuesto. Sincronizar etiquetas, fechas y subtareas con GitHub.
Audio dinámico y controles	En refinamiento / no verificado	Especificación aportada por el Scrum Master y prototipo con base de SFX/controles. Verificar BGM procedural, ambiente y sliders en la versión publicada; no declarar implementación Unity sin evidencia.

Plataforma y alcance

La ERS define Unity 2022.3 LTS, C# y el nuevo Input System. El alcance demostrado por los controles es de escritorio con teclado y mouse. Controles táctiles y consumo de batería quedan fuera de esta entrega mientras no se acuerde una versión móvil. La visualización del prototipo en navegador no acredita una build web o móvil del juego.

9.1 Mediciones de calidad pendientes

Se conservan los umbrales de la ERS. Estos criterios definen cómo comprobar la calidad y no son mediciones obtenidas del prototipo ni del tablero.

Atributo y objetivo	Procedimiento propuesto	Aceptación y estado
RNF-01 Rendimiento Promedio mínimo de 60 FPS con hasta 50 enemigos.	En build de escritorio, registrar CPU, GPU, RAM y resolución. Probar con 10, 25 y 50 enemigos durante 60 s; repetir tres veces, excluyendo carga inicial.	Promedio ≥60 FPS en cada carga. Guardar mediciones y condiciones. Pendiente de ejecución.
RNF-02 Usabilidad 90 % reconoce vida, estamina y oleada en menos de 60 s.	Prueba sin explicación previa con 10 participantes propuestos. Cronometrar la identificación de los tres elementos; registrar aciertos y observaciones.	Al menos 9 de 10 identifican los tres en menos de 60 s. Reportar tamaño de muestra. Pendiente.
RNF-03 Respuesta No superar 100 ms.	Instrumentar tiempo entre entrada y respuesta visible de movimiento, ataque, arma y esquiva. Registrar al menos 30 muestras por acción y condiciones de carga.	Ninguna muestra >100 ms en el ensayo; informar máximo y distribución, sin generalizar fuera de las condiciones probadas. Pendiente.
Confiabilidad Sesión y reinicios estables.	Propuesta: sesión de 15 min y 10 reinicios consecutivos, incluyendo pausa y resultados.	Sin cierre inesperado, bloqueo, conteos duplicados ni estados persistentes de partida previa. No verificado.
Portabilidad y legibilidad Entorno de entrega.	Ejecutar la build en el equipo objetivo y en un segundo equipo; probar 1280×720 y 1920×1080.	Controles disponibles, textos legibles, menús sin recortes y flujo completo. No verificado.

Gestión de defectos y recursos

Hefer coordina la evidencia de QA; Gilmar apoya las sesiones de validación. Cada defecto se vincula al requisito, build y pasos de reproducción. La licencia de los assets se revisa conforme a la ERS, que exige recursos gratuitos o creados por el equipo; el cumplimiento de todos los recursos todavía debe comprobarse.

10 Riesgos y reto de integración

Riesgo	Prevención	Prueba asociada
Muerte duplicada y oleada bloqueada	Identificadores únicos, muerte emitida una vez y conjunto de enemigos activos.	P01 y P02.
Acciones o mejoras siguen en pausa	Estado central y tiempo de juego compartido para ataques, IA, generación y duración.	P03.
Referencias de escena y eventos acumulados	Configurar dependencias al cargar y retirar suscripciones al cerrar o reiniciar.	P03 y P05.
HUD diferente del estado real	Eventos con valores actuales; UI sin cambios directos sobre vida o estamina.	Paso 06 y RNF-02.
Fechas o estados inconsistentes	Sincronizar calendario propuesto con tablero y actualizar avances mediante evidencia.	Revisión semanal del Sprint Planning.

Aporte para el foro de participación

Nuestro mayor reto de integración en Dead Harvest es coordinar combate, enemigos y oleadas para que una eliminación se registre una sola vez y produzca la transición correcta. Un enemigo puede recibir varios impactos en un intervalo breve; si notifica su muerte más de una vez, el contador puede quedar incorrecto o terminar la ronda antes de tiempo.

Proponemos contratos de daño, identificadores únicos y un evento EnemigoEliminado con enemigoId y oleadaId. Oleadas mantendrá un conjunto de enemigos activos y solo descontará un ID registrado. Integraremos primero jugador y combate, después enemigos y finalmente oleadas. Tras cada paso repetiremos las pruebas anteriores y registraremos el resultado. Esta estrategia permite localizar el fallo en el último cambio sin integrar todos los sistemas al final.

Evidencia y alcance de esta versión

Esta entrega completa el diseño técnico y el plan de comprobación. La copia del repositorio revisada contiene documentación y diseño; no incluye un proyecto Unity que permita verificar ejecución. Los avances que el equipo tenga en otra ubicación deberán incorporarse con su versión y evidencia antes de cambiar los estados de calidad a cumplido.

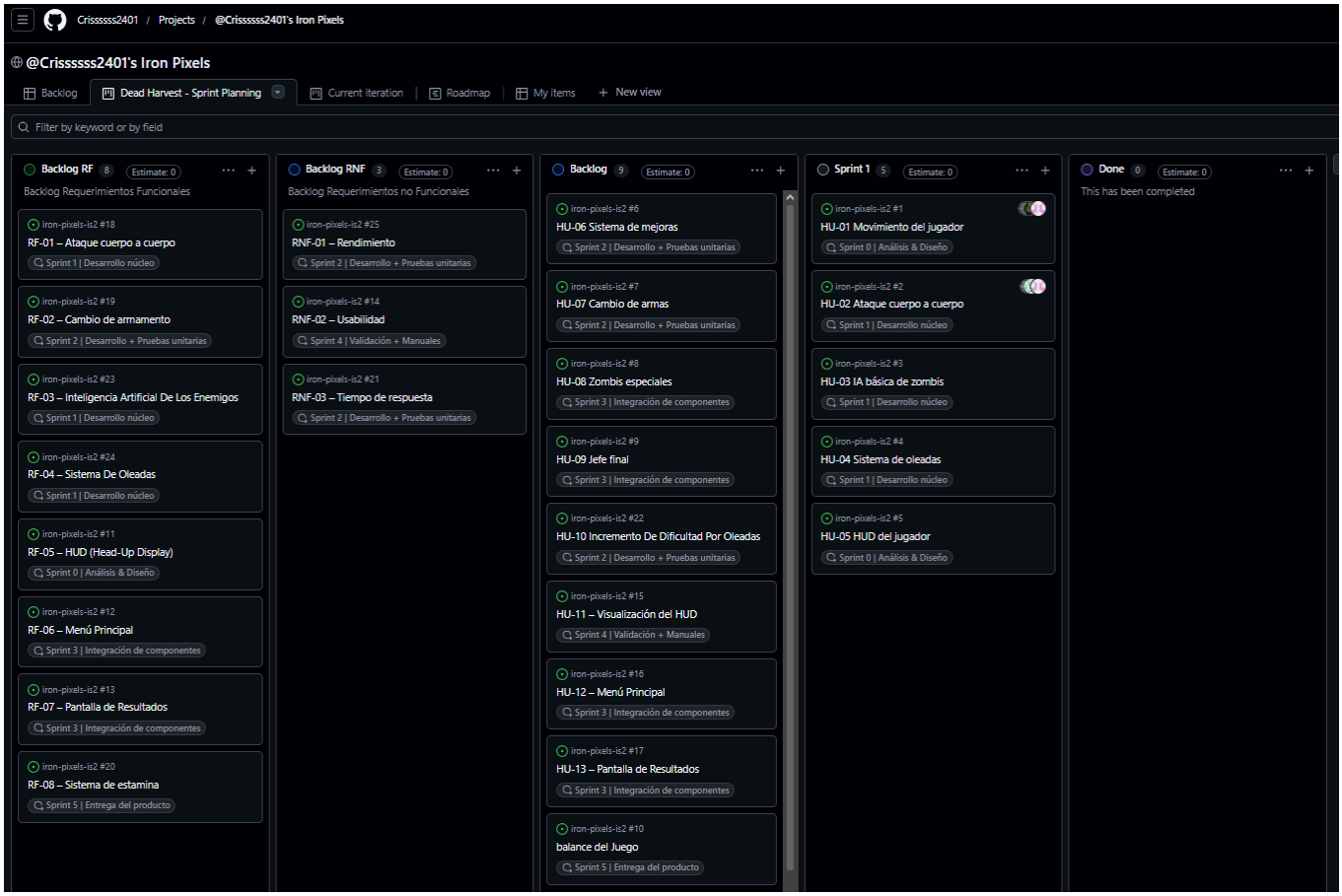
10.1 Control de coherencia con la ERS

La ERS v0200 define el MVP en Unity 2D con movimiento, cuatro armas cuerpo a cuerpo, IA por FSM, oleadas, HUD, menú, resultados, estamina, botiquines y jefe final; también deja fuera el guardado persistente. El diseño C01-C10 mantiene ese alcance. Audio se conserva como componente de soporte/presentación porque la ERS exige feedback sonoro en RF-01 y ya identifica un subsistema de Audio y Efectos. Las ampliaciones musicales del prototipo no se convierten en requisitos funcionales nuevos en esta entrega.

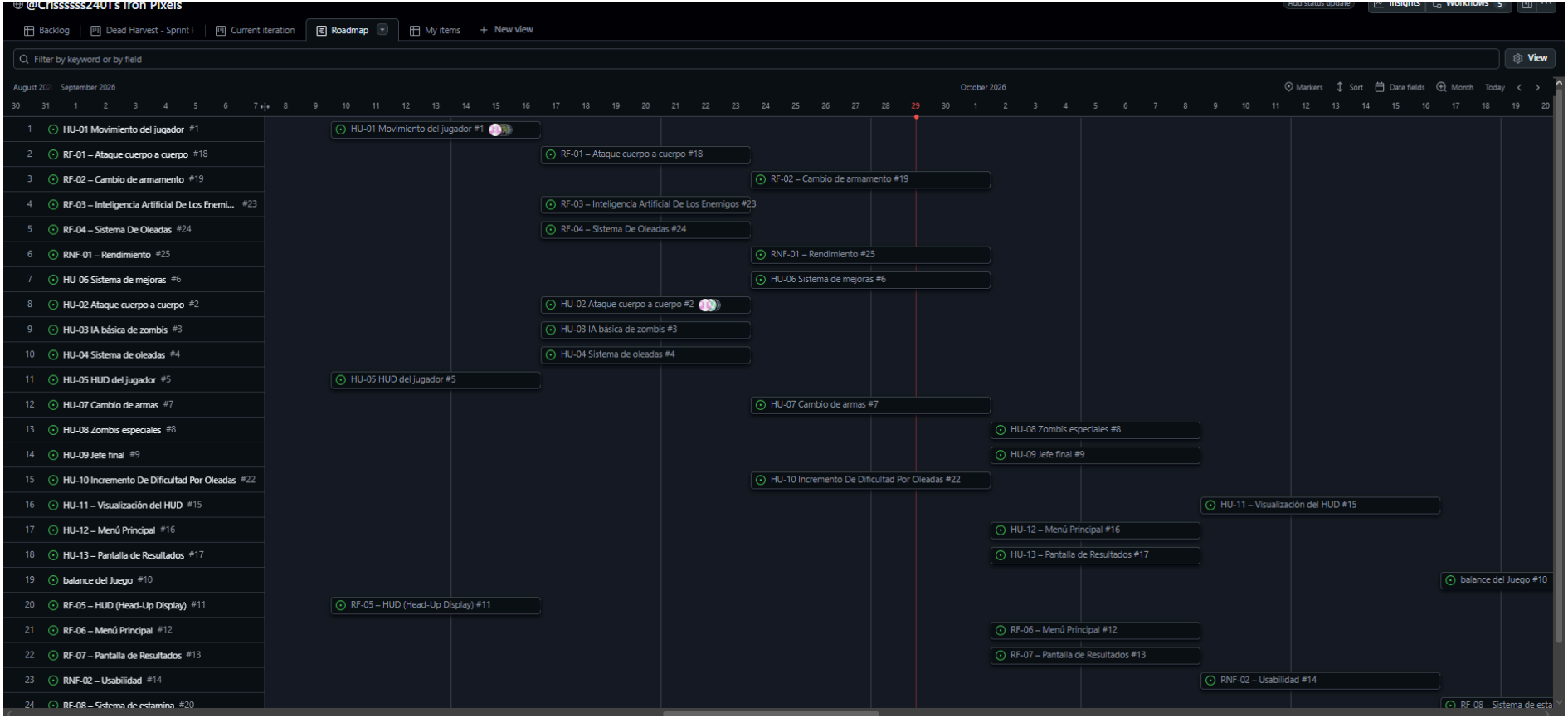
11 Fuentes y anexos de planificación

Fuente	Uso en esta entrega
Actividad de Canvas del 29 de septiembre de 2026	Alcance: cronograma de sprints, enlace del prototipo y componentes de la clase 10.
Clase 10 Componentes Calidad y Semana 11 Material Estudiante	Responsabilidad única, interfaces, integración incremental, checklist y foro.
Dead Harvest Planificación y cronograma de Sprints de trabajo PDF	Antecedente de planificación, 65 actividades, responsables y hallazgos UX.
Dead Harvest Sprint Planning de Hefer Cotuc XLSX	Detalle de tareas, responsables, criterios de terminado y fuentes.
Repositorio iron-pixels-is2 adjunto	ERS-04, ERS-05, diagrama de componentes de Semana 6, wireframes y mockups de Semana 7.
Capturas de GitHub Projects	Tablero y Roadmap originales, reproducidos en las páginas siguientes.
Informes de Gilmar y Nahum	Ocho hallazgos heurísticos, conservados como pendientes de comprobación de correcciones.
ERS COMPLETA v0200 corregida	Referencia de alcance y trazabilidad: RF-01 a RF-08, RNF-01 a RNF-03, restricciones Unity 2022.3 LTS/C#, botiquines y exclusión de guardado persistente.

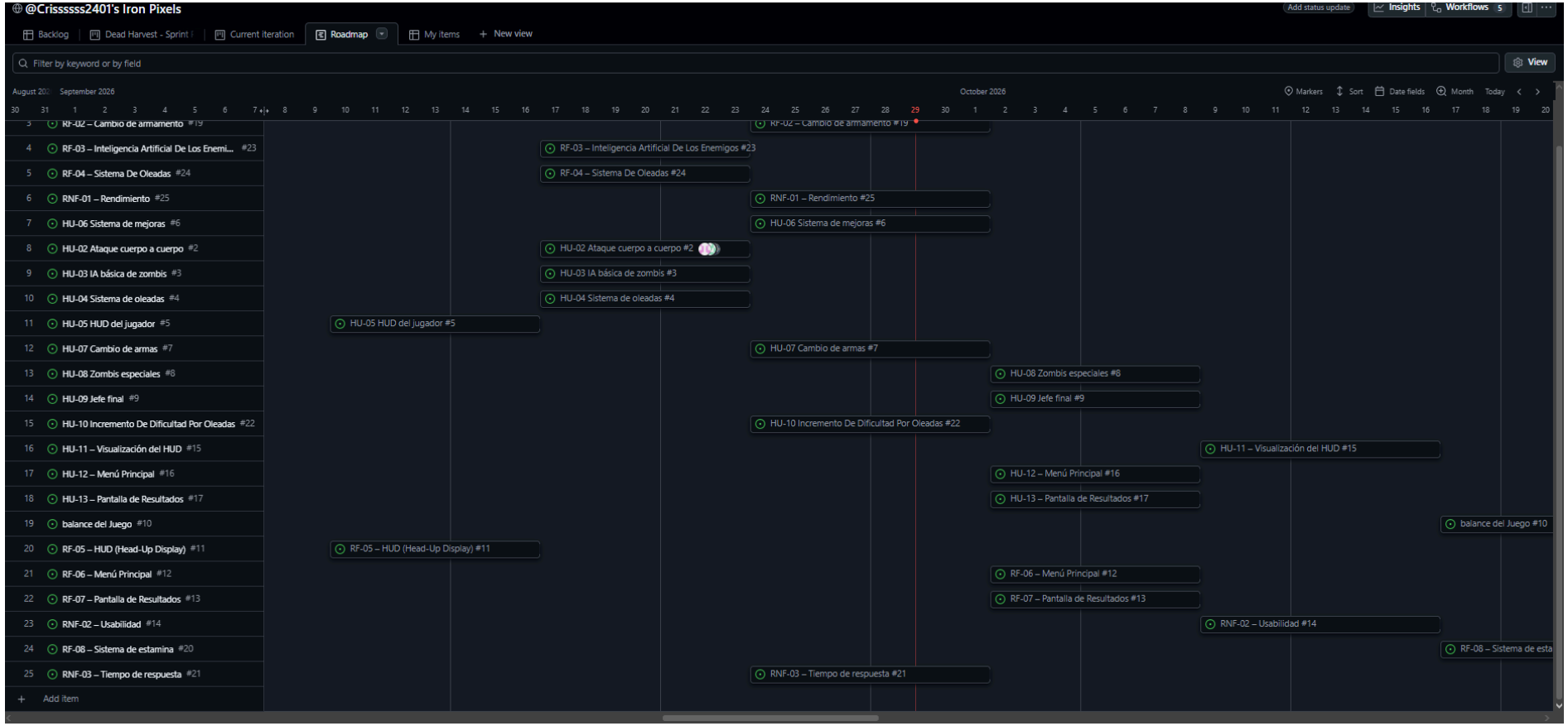
Anexo A Sprint Planning existente



Anexo B Roadmap de septiembre y octubre



Anexo C Continuación del Roadmap



Anexo D Detalle de octubre

